

```

24
25 -- 2. View complete product catalog with all available details
26 • select *
27 from product_reviews;
28
29 -- 3. List all unique product categories

```

Result Grid |  Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	review_id	product_id	customer_id	rating	review_text	review_date
▶	1	1	21	2	Great product, loved it!	2024-11-07 02:40:51
	2	9	27	4	Product arrived damaged, had to return.	2025-06-08 02:40:51
	3	42	21	5	Very fast delivery, happy with the purchase.	2024-10-01 02:40:51
	4	13	28	2	Average quality, but acceptable.	2024-12-23 02:40:51
	5	45	9	4	Good value for money.	2024-10-07 02:40:51
	6	47	30	5	Product arrived damaged, had to return.	2024-07-06 02:40:51
	7	6	25	4	Product arrived damaged, had to return.	2024-11-10 02:40:51
	8	27	29	5	Great product, loved it!	2024-08-02 02:40:51
	9	5	13	1	Average quality, but acceptable.	2024-11-09 02:40:51
	10	49	23	3	Product arrived damaged. had to return.	2025-05-19 02:40:51

product_reviews 55 x

```
28
29 -- 3. List all unique product categories
30 • select distinct category
31 from products;
32
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	category
▶	Home
	Clothing
	Electronics
	Toys
	Books

products 56 x

```

32
33 -- 4. Show all products priced above ₹1,000
34 • select * from products
35 where price > 1000;

```

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 						
	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38

products 58 ×

Output

```

36
37 -- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)
38 • select * from products
39 where price between 2000 and 5000;
40
41 -- 6. Fetch data for specific customer IDs (e.g., from loyalty program list)

```

Result Grid |   Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55

products 59 ×

```
40
41 -- 6. Fetch data for specific customer IDs (e.g., from loyalty program list)
42 • select * from customers
43 where customer_id in (1,3,5,8);
44
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	customer_id	name	email	phone	created_at
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
	3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
	5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
	8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
•	NULL	NULL	NULL	NULL	NULL

customers 60 x

```

44
45 -- 7. Identify customers whose names start with the letter 'A
46 • select * from customers
47 where name like 'A%';

```

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 					
	customer_id	name	email	phone	created_at
▶	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
•	NULL	NULL	NULL	NULL	NULL

```

49 -- 8. List electronics products priced under ₹3,000
50 • select * from products
51 where category = 'electronics' and price < 3000;
52
53 -- 9. Display product names and prices in descending order of price

```

Result Grid



Filter Rows:

Edit:



Export/Import:



Wrap Cell C

	product_id	name	category	price	stock_quantity	added_on
▶	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
	47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
•	NULL	NULL	NULL	NULL	NULL	NULL

products 62 ×

Output



Search



```

53      -- 9. Display product names and prices in descending order of price
54 •    select name as product_names, price
55      from products
56      order by price desc;
57
58      -- 10. Display product names and prices, sorted by price and then by name
59 •    select name as product_names, price

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	product_names	price
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66
	Build High	4707.14
	Serious Recognize	4523.10
	Study Total	4413.68
	Real Source	4398.66

products 63 ×

```
57
58 -- 10. Display product names and prices, sorted by price and then by name
59 • select name as product_names, price
60 from products
61 order by price, product_names;
62
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	product_names	price
▶	If Whatever	177.40
	Listen Development	296.17
	Actually Term	396.11
	Television Stock	421.73
	Southern Thing	512.46
	Plant No	639.43
	Place Low	723.97
	Stock Article	834.75
	Toward Minute	857.85
	Common Studv	985.19

products 64 x

```
64 -- Level 2: Filtering and Formatting
65
66 -- 1. Retrieve orders where customer information is missing (possibly due to data migration ordeletion)
67 • select * from orders
68 where customer_id is null;
69
70 -- 2. Display customer names and emails using column aliases for frontend readability
71 • select name as customer_name, email as email_address
```

Result Grid |   Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	order_id	customer_id	order_date	status	total_amount
•	NULL	NULL	NULL	NULL	NULL

```
69
70 -- 2. Display customer names and emails using column aliases for frontend readability
71 • select name as customer_name, email as email_address
72    from customers;
73
74 -- 3. Calculate total value per item ordered by multiplying quantity and item price
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	customer_name	email_address
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com

customers 66 ×

Output

```

74  -- 3. Calculate total value per item ordered by multiplying quantity and item price
75  •  select order_item_id, product_id, sum(quantity * item_price) as total_value
76  from order_items
77  group by order_item_id, product_id;
78
79  -- 4. Combine customer name and phone number in a single column
80  •  select concat(name, '-', phone) as customer_name_contact

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	order_item_id	product_id	total_value
▶	1	19	9414.28
	2	6	532.20
	3	9	2955.57
	4	23	2208.99
	5	35	9469.78
	6	19	4707.14
	7	7	9794.72
	8	7	14692.08
	9	37	1429.45
	10	15	723.97

Result 67 ×

Output

```

70
79 -- 4. Combine customer name and phone number in a single column
80 • select concat(name, '-', phone) as customer_name_contact
81 from customers;
82
83 -- 5. Extract only the date part from order timestamps for date-wise reporting

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	customer_name_contact
▶	Thomas Owens-142-479-1945
	Charles Grant-9153947511
	Kaitlin Richards-2073473421
	Christina Williams-586-605-5061x06
	David Allen-(751)456-8289x1
	Mark Duke-(144)957-2811
	Briana Wright-223-833-9635
	John Bryan-045.568.0798x27
	Jason Thompson-1862659420
	Shawn Hill-(268)113-3152x7

Result 68 ×

Output

```
82
83 -- 5. Extract only the date part from order timestamps for date-wise reporting
84 • select order_id, date(order_date) as date
85 from orders;
86
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	order_id	date
▶	1	2025-03-02
	2	2024-10-09
	3	2025-05-08
	4	2024-09-19
	5	2025-04-08
	6	2024-10-25
	7	2024-07-29
	8	2024-07-30
	9	2025-06-10
	10	2025-02-16

Result 70 x

```
86
87 -- 6. List products that do not have any stock left
88 • select product_id, stock_quantity
89 from products
90 where stock_quantity = 0;
91
92 -- Level 3: Aggregations
```

Result Grid



Filter Rows:

Edit:



Export/Import:



Wrap Cell

	product_id	stock_quantity
•	NULL	NULL

```
91
92 -- Level 3: Aggregations
93
94 -- 1. Count the total number of orders placed
95 • select count(status) as order_placed
96 from orders;
97
98 -- 2. Calculate the total revenue collected from all orders
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	order_placed
▶	400

```
97
98 -- 2. Calculate the total revenue collected from all orders
99 • select sum(total_amount) as total_revenue
100 from orders;
101
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	total_revenue
▶	6960973.66

101

102

-- 3. Calculate the average order value

103 •

select avg(total_amount) as avg_order_value

104

from orders;

105

106

-- 4. Count the number of customers who have placed at

107 •

select count(distinct customer_id)

Result Grid



Filter Rows:

Export:



Wrap Cell Center

	avg_order_value
▶	17402.434150

```
105
106 -- 4. Count the number of customers who have placed at least one order
107 • select count(distinct customer_id)
108    from orders;
109
110 -- 5. Find the number of orders placed by each customer
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	count(distinct customer_id)
▶	30

109

110 -- 5. Find the number of orders placed by each customer

111 • select customer_id, count(order_id) as total_orders

112 from orders

113 group by customer_id;

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	customer_id	total_orders
▶	1	12
	2	17
	3	17
	4	9
	5	14
	6	16
	7	13
	8	10
	9	10
	10	13

Result 76 ×

Output

```
115 -- 6. Find total sales amount made by each customer
116 • select customer_id, sum(total_amount) as total_sales
117 from orders
118 group by customer_id;
119
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	customer_id	total_sales
▶	1	183747.44
	2	284420.07
	3	253783.31
	4	137562.22
	5	262504.19
	6	212173.58
	7	167960.11
	8	164701.78
	9	106226.03
	10	252722.75

Result 77 ×

Output

```
120 -- 7. List the number of products sold per category
121 • select p.category, sum(oi.quantity)
122 from products as p
123 inner join order_items oi
124 on p.product_id = oi.product_id
125 group by category;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	category	sum(oi.quantity)
►	Clothing	89
	Electronics	73
	Toys	56
	Home	62
	Books	30

```
127 -- 8. Find the average item price per category
128 • select category, avg(price) as avg_price
129 from products
130 group by category;
131
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	category	avg_price
▶	Home	2146.367500
	Clothing	3434.581538
	Electronics	2653.388571
	Toys	2516.526250
	Books	3167.084286

```
131
132      -- 9. Show number of orders placed per day
133 •   select date(order_date) as day, count(*) as orders
134      from orders
135      group by day;
136
137      -- 10. List total payments received per payment method
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	day	orders
▶	2025-03-02	3
	2024-10-09	2
	2025-05-08	2
	2024-09-19	3
	2025-04-08	3
	2024-10-25	3
	2024-07-29	2
	2024-07-30	2
	2025-06-10	2
	2025-02-16	2

Result 80 ×

Output

```
136
137 -- 10. List total payments received per payment method
138 • select method, sum(amount_paid) as total_payments
139 from payments
140 group by method;
141
142
143 -- Level 4: Multi-Table Queries (JOINS)
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	method	total_payments
▶	Credit Card	1754603.10
	Net Banking	1658383.90
	UPI	1617408.78
	Debit Card	1930577.88

144

145 -- 1. Retrieve order details along with the customer name

146 • select o.order_id, c.customer_id, name, o.order_date, o.total_amount

147 from orders as o

148 inner join customers as c

149 on c.customer_id = o.customer_id;

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	order_id	customer_id	name	order_date	total_amount
▶	14	1	Thomas Owens	2024-09-24 21:21:38	15803.34
	17	1	Thomas Owens	2024-08-19 21:17:57	11173.77
	61	1	Thomas Owens	2024-12-26 23:21:58	13053.00
	76	1	Thomas Owens	2025-01-14 22:59:54	23506.81
	92	1	Thomas Owens	2024-09-26 11:33:58	834.75
	109	1	Thomas Owens	2025-03-17 04:56:18	12190.14
	127	1	Thomas Owens	2025-06-10 18:01:32	20160.64
	135	1	Thomas Owens	2025-03-11 22:58:23	8891.79
	144	1	Thomas Owens	2024-09-19 09:18:22	12093.54
	221	1	Thomas Owens	2024-09-25 05:49:39	12437.61

Result 82 x

Output

```

151 -- 2. Get list of products that have been sold (INNER JOIN with order_items
152 • select distinct p.product_id, p.name
153 from products as p
154 inner join order_items as oi
155 on oi.product_id = p.product_id;
156

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	product_id	name
▶	1	Plant No
	2	Population Social
	4	Any Question
	5	Natural Network
	6	If Whatever
	7	Response Indeed
	8	Every Amount
	9	Common Study
	10	Development System
	11	Build Her

Result 84 ×

```

156
157 -- 3. List all orders with their payment method (INNER JOIN)
158 • select o.order_id, p.method, p.amount_paid
159 from orders as o
160 inner join payments as p
161 on o.order_id = p.order_id;
162

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	order_id	method	amount_paid
▶	14	UPI	15803.34
	17	UPI	11173.77
	61	Net Banking	13053.00
	76	Credit Card	23506.81
	92	Net Banking	834.75
	109	UPI	12190.14
	127	Debit Card	20160.64
	135	Debit Card	8891.79
	144	UPI	12093.54
	221	UPI	12437.61

Result 85 ×

162

163 -- 4. Get list of customers and their orders (LEFT JOIN)

164 • select c.name, o.order_id, o.order_date, o.status, o.total_amount

165 from customers as c

166 inner join orders as o

167 on c.customer_id = o.customer_id;

168

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	name	order_id	order_date	status	total_amount
▶	Thomas Owens	14	2024-09-24 21:21:38	Shipped	15803.34
	Thomas Owens	17	2024-08-19 21:17:57	Pending	11173.77
	Thomas Owens	61	2024-12-26 23:21:58	Shipped	13053.00
	Thomas Owens	76	2025-01-14 22:59:54	Pending	23506.81
	Thomas Owens	92	2024-09-26 11:33:58	Shipped	834.75
	Thomas Owens	109	2025-03-17 04:56:18	Shipped	12190.14
	Thomas Owens	127	2025-06-10 18:01:32	Pending	20160.64
	Thomas Owens	135	2025-03-11 22:58:23	Pending	8891.79
	Thomas Owens	144	2024-09-19 09:18:22	Shipped	12093.54
	Thomas Owens	221	2024-09-25 05:49:39	Pending	12437.61

Result 86 ×

Output

```

168
169 -- 5. List all products along with order item quantity (LEFT JOIN)
170 • select p.name, oi.quantity
171 from products as p
172 left join order_items as oi
173 on oi.product_id = p.product_id;
174

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	name	quantity
▶	Plant No	1
	Plant No	3
	Plant No	1
	Plant No	1
	Plant No	2
	Population Social	2
	Population Social	2
	Population Social	2
	Available Answer	NULL
	Any Question	3

Result 87 ×

```

174
175 -- 6. List all payments including those with no matching orders (RIGHT JOIN)
176 • select o.order_id, p.payment_id, p.amount_paid
177 from payments as p
178 right join orders as o
179 on p.order_id = o.order_id;
180

```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	order_id	payment_id	amount_paid
▶	14	14	15803.34
	17	17	11173.77
	61	61	13053.00
	76	76	23506.81
	92	92	834.75
	109	109	12190.14
	127	127	20160.64
	135	135	8891.79
	144	144	12093.54
	221	221	12437.61

Result 88 ×

```

180      Execute the selected portion of the script or everything, if there is no selection
181      -- 7. Combine data from three tables: customer, order, and payment
182      select *
183      from customers as c
184      inner join orders as o
185      on c.customer_id = o.customer_id
186      inner join payments as p
187      on p.order_id = o.order_id;
188
189

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount	payment_id	ord
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	1	2024-09-24 21:21:38	Shipped	15803.34	14	14
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	1	2024-08-19 21:17:57	Pending	11173.77	17	17
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	1	2024-12-26 23:21:58	Shipped	13053.00	61	61
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	1	2025-01-14 22:59:54	Pending	23506.81	76	76
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	1	2024-09-26 11:33:58	Shipped	834.75	92	92
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	1	2025-03-17 04:56:18	Shipped	12190.14	109	109
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	1	2025-06-10 18:01:32	Pending	20160.64	127	127
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	1	2025-03-11 22:58:23	Pending	8891.79	135	135
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	144	1	2024-09-19 09:18:22	Shipped	12093.54	144	144

Result 89 x

Read

Output

```

190 -- Level 5: Subqueries (Inner Queries)
191
192 -- 1. List all products priced above the average product price
193 • select *
194 from products
195 where price > (select avg(price) from products);
196
197 -- 2. Find customers who have placed at least one order
198 • select c.customer_id, c.name, c.email

```

Result Grid |   Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27

products 90 x

```

196
197 -- 2. Find customers who have placed at least one order
198 • select c.customer_id, c.name, c.email
199 from customers as c
200 where exists (select order_id
201 from orders as o
202 where o.customer_id = c.customer_id);
203
204 -- 3. Show orders whose total amount is above the average for that customer

```

Result Grid



Filter Rows:

Edit:



Export/Import:



Wrap Cell Content:



	customer_id	name	email
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	3	Kaitlin Richards	user3@example.com
	4	Christina Williams	user4@example.com
	5	David Allen	user5@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	8	John Bryan	user8@example.com
	9	Jason Thompson	user9@example.com
	10	Shawn Hill	user10@example.com

customers 91 x

Output

```

203
204 -- 3. Show orders whose total amount is above the average for that customer
205 • select *
206 from orders
207 where total_amount > (select avg(total_amount) as total_amount
208 from orders as o
209 where customer_id = o.customer_id);
210

```

Result Grid |  Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	order_id	customer_id	order_date	status	total_amount
▶	6	29	2024-10-25 07:33:59	Cancelled	31050.17
	8	19	2024-07-30 22:49:49	Cancelled	32714.06
	9	6	2025-06-10 17:00:25	Pending	24219.20
	10	28	2025-02-16 12:45:59	Delivered	24342.52
	16	5	2024-09-09 03:00:40	Shipped	22856.73
	18	13	2024-06-15 12:57:04	Cancelled	32001.24
	21	22	2025-03-12 18:53:46	Pending	25364.11
	22	20	2025-02-21 11:21:06	Shipped	21281.44
	23	25	2025-03-02 14:33:23	Delivered	18045.05
	24	6	2024-06-19 07:09:04	Pending	22882.19

orders 92 ×

Output

210

211 -- 4. Display customers who haven't placed any orders

212 • select *

213 from customers

214 where customer_id not in (select customer_id

215 from orders);

216

Result Grid



Filter Rows:

Edit:



Export/Import:



	customer_id	name	email	phone	created_at
•	NULL	NULL	NULL	NULL	NULL

```

216
217 -- 5. Show products that were never ordered
218 • select *
219 from products
220 where product_id not in (select product_id
221 from order_items);
222

```

Result Grid



Filter Rows:

Edit:



Export/Import:



Wrap

	product_id	name	category	price	stock_quantity	added_on
▶	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
•	NULL	NULL	NULL	NULL	NULL	NULL

```

223      -- 6. Show highest value order per customer
224 •    select *
225      from orders
226      where total_amount = (select max(total_amount)
227      from orders as o
228      where customer_id = o.customer_id);
229
230      -- 7. Highest Order Per Customer (Including Names)
231 •    select c.name, o.order_id, o.total_amount

```

Result Grid		Filter Rows:	Edit: 			Export/Import: 		Wrap Cell Content: 
	order_id	customer_id	order_date	status	total_amount			
▶	107	14	2025-02-01 02:02:18	Shipped	48042.06			
•	NULL	NULL	NULL	NULL	NULL			

```

230 -- 7. Highest Order Per Customer (Including Names)
231 • Select customer_name, o.order_id, o.total_amount
232 from orders as o
233 inner join customers as c
234 on c.customer_id = o.customer_id
235 where total_amount = (select max(total_amount)
236 from orders
237 where customer_id = o.customer_id);
238
239
240 -- Level 6: Set Operations

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	name	order_id	total_amount
▶	Thomas Owens	278	32015.16
	Charles Grant	315	42056.04
	Kaitlin Richards	281	41679.11
	Christina Williams	386	25747.34
	David Allen	266	39921.78
	Mark Duke	330	39003.19
	Briana Wright	128	28589.04
	John Bryan	33	36147.09
	Jason Thompson	348	21414.44
	Shawn Hill	62	35723.17

Result 96 x

Output

```

241
242 -- 1. List all customers who have either placed an order or written a product review
243 • select customer_id, name, email
244   from customers
245   where customer_id in (select customer_id
246                        from orders
247                        union
248                        select customer_id
249                        from product_reviews);
250
251 -- 2. List all customers who have placed an order as well as reviewed a product [intersect notsupported]

```

Result Grid |  Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	customer_id	name	email
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	3	Kaitlin Richards	user3@example.com
	4	Christina Williams	user4@example.com
	5	David Allen	user5@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	8	John Bryan	user8@example.com
	9	Jason Thompson	user9@example.com
	10	Shawn Hill	user10@example.com

customers 97 x

Output

```

250
251 -- 2. List all customers who have placed an order as well as reviewed a product [intersect notsupported]
252 • select name
253 from customers
254 where customer_id in (select customer_id
255 from orders
256 where customer_id in (select customer_id
257 from product_reviews));
258
259

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	name
▶	Thomas Owens
	Charles Grant
	Christina Williams
	Mark Duke
	Briana Wright
	Jason Thompson
	Shawn Hill
	Walter Jenkins
	Leslie Wilson
	Deborah Arias

customers 99 x

Output

erce_db

es
customers
order_items
orders
payments
product_reviews
Schemas

e_db

```
16
17 • USE ecommerce_db;
18
19 -- Level 1: Basics
20
21 -- 1. Retrieve customer names and emails for email marketing
22 • select name as customer_name, email
23 from customers;
24
25 -- 2. View complete product catalog with all available details
26 • select *
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	customer_name	email
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com

customers 54 x